Information Security

Lecture 5: Public-Key Cryptography, Integrity and Trust

A textbook treatment

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **2**

5 Public-Key Cryptography, Integrity and Trust **3**
5.1 Why Asymmetric 3
5.2 Number Theory for RSA 3
5.3 RSA 4
5.4 Hash Functions and Passwords 5
5.5 MACs and Digital Signatures 6
5.6 Certificates and PKI 7
5.7 Steganography 8

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 5

Public-Key Cryptography, Integrity and Trust

5.1 Why Asymmetric

With symmetric keys, N people need $\binom{N}{2} = N(N-1)/2$ keys (6 people: 15) and a secure channel for each; with key pairs, $2N$ (6 people: 12), and no pre-shared secret. The mailbox principle: anyone can drop a letter, only the owner can open the box. Diffie, Hellman and Merkle published the idea in 1976.

Definition 5.1.1 (One-way and trapdoor functions; requirements)**.** A **one-way function** $y = f(x)$ is easy to compute and infeasible to invert. A **trapdoor function** is one-way except to whoever holds secret information t (generated with f by an algorithm Gen); the public key is f, the private key is t. A public-key system must make key generation, encryption with the public key and decryption with the private key *easy*, and deriving the private key from the public key, or the message from the ciphertext, *infeasible*. Candidate: factoring; $6895601 = 1931 \times 3571$ is hard to find and trivial to check given $t = 1931$.

Key insight

Public keys remove the need for a *secret* channel but create the need for an *authentic* one: Alice must know that the public key is Bob's, or a man in the middle substitutes his own. Certificates (§5.6) exist for this.

5.2 Number Theory for RSA

Definition 5.2.1 (Modular arithmetic, φ, inverses)**.** $a \equiv b \pmod{n}$ iff $n \mid a-b$. $\gcd(a,n) = 1$ iff a has a multiplicative inverse mod n, found by the extended Euclidean algorithm (for tiny numbers: try $d = 1, 2, \ldots$ until $ed \bmod \varphi = 1$). Euler's totient $\varphi(n)$ counts $1 \le a < n$ coprime to n; for distinct primes $\varphi(pq) = (p-1)(q-1)$. **Euler's theorem**: $\gcd(a,n) = 1 \Rightarrow a^{\varphi(n)} \equiv 1 \pmod{n}$. Fast exponentiation: square repeatedly and multiply the needed powers (the slide's $26^2, 26^4$, then $26^7 = 26^4 \cdot 26^2 \cdot 26$).

5.3 RSA

Definition 5.3.1 (RSA (Rivest, Shamir, Adleman, 1977))**.** Choose distinct large primes p, q (random, similar size); $n = pq$; $\varphi = (p-1)(q-1)$; choose e with $1 < e < \varphi$, $\gcd(e, \varphi) = 1$; compute d with $ed \equiv 1 \pmod{\varphi}$. Public key (n, e), private key (n, d); p, q, φ stay secret. Encrypt $c = m^e \bmod n$; decrypt $m = c^d \bmod n$.

Theorem 5.3.2 (Correctness)**.** *For* $0 \le m < n$, $(m^e)^d \equiv m \pmod{n}$.

Proof. $ed = 1 + k\varphi$ for some integer k. If $\gcd(m, n) = 1$, Euler gives $m^{ed} = m \cdot (m^{\varphi})^k \equiv m$. If $p \mid m$, both sides are $0 \bmod p$, and mod q Fermat gives $m^{ed} \equiv m \cdot (m^{q-1})^{k(p-1)} \equiv m$; by the Chinese remainder theorem the congruence holds mod n (symmetrically for $q \mid m$). □

Worked example: Slide keys and the TP password

Slide: $p = 3, q = 11$, $n = 33$, $\varphi = 20$, $e = 3$, $d = 7$ $(21 \equiv 1)$; $m = 4$: $c = 64 \bmod 33 = 31$, $31^7 \bmod 33 = 4$; $m = 5$: $c = 125 \bmod 33 = 26$, $26^2 \equiv 16$, $26^4 \equiv 25$, $26^7 \equiv 25 \cdot 16 \cdot 26 \equiv 5$. TP: $p = 5, q = 11, e = 3, C = 23$: $n = 55$, $\varphi = 40$, $d = 27$ $(81 = 2 \cdot 40 + 1)$; $23^2 \equiv 34$, $23^4 \equiv 34^2 = 1156 \equiv 1$, so $23^{27} = (23^4)^6 \cdot 23^3 \equiv 23 \cdot 34 = 782 \equiv 12$: password $m = 12$; check $12^3 = 1728 = 31 \cdot 55 + 23$. Fresh pair: $p = 7, q = 13, e = 5$: $n = 91, \varphi = 72, d = 29$; $m = 2 \mapsto c = 32$.

Interpretation & discussion

Security rests on factoring n: with p, q one computes φ and d. The RSA Factoring Challenge shows the frontier: RSA-768 (232 digits) factored December 2009, RSA-250 (829 bits) February 2020; 1024-bit keys "are out there", 2048 is the default. A 50-bit modulus (TP) falls to trial division up to $\sqrt{n} \approx 2^{25}$ in seconds; a 512-bit one needs the number field sieve (deprecated, but not trial division). RSA is slow and the message must be smaller than n: encrypt a symmetric key, not a file (*hybrid*: random AES key and IV, message under AES-256-CBC, key under the recipient's RSA public key). Textbook RSA is deterministic and malleable: a 4-digit PIN is found by encrypting all 10^4 candidates and comparing; real systems use randomised padding (OAEP).

From the TP: OpenSSL mission 3

`openssl genrsa -out k.pem 512` writes an *unencrypted* private key (anyone who reads the file owns it); `openssl rsa -in k.pem -text -noout` shows n, e, d, p, q; `-pubout` extracts the public key, which may be stored in plaintext; `-aes256` at generation encrypts the key file under a passphrase with AES-256-CBC; `pkeyutl -encrypt -pubin` works on a short file and fails on a large one. Factoring the 50-bit `otherSmall_pub.pem`: extract n with `-pubin -text`, trial-divide to $\sqrt{n}$, compute d.

Go deeper: the details an examiner asks about

- e must be coprime to φ or d does not exist; common $e = 65537 = 2^{16}+1$.

- Primes "similar in magnitude": a small prime is found by trial division; primes too close fall to Fermat's method $n = a^2 - b^2$; $p = q$ makes n a square.
- The slide says "modulo a prime"; precisely, modulo $n = pq$.

Hard questions, answered

Q. Why is combining AES and RSA better than RSA alone for a long message?
A. RSA is slow and bounded by n; AES is fast and unbounded. RSA protects only the 32-byte key; AES does the bulk; no pre-shared secret is needed.

Q. Eve knows (n, e) and the ciphertext of a 4-digit PIN. Attack?
A. Encrypt all 10^4 PINs under (n, e) and compare: textbook RSA is deterministic. Fix: OAEP padding.

5.4 Hash Functions and Passwords

Definition 5.4.1 (Cryptographic hash)**.** H maps arbitrary-length input to a fixed-length digest, fast, lossy, with a tiny input change producing a very different digest. Requirements: **preimage resistance** (given y, hard to find x with $H(x) = y$), **second-preimage resistance** (given x, hard to find $x' \neq x$ with $H(x') = H(x)$), **collision resistance** (hard to find *any* $m_1 \neq m_2$ with equal digests). Collisions exist (finite outputs); the question is the work to find one: generic preimages cost $\approx 2^n$, collisions $\approx 2^{n/2}$ (birthday bound). Collision resistance implies second-preimage resistance; neither implies preimage resistance. MD5 and SHA-1 have practical collisions; SHA-2 (224/256/384/512) is current.

Theorem 5.4.2 (A hash alone gives no authenticity)**.** *If Alice sends $(m, H(m))$ over a channel Eve controls, Eve can replace it by $(m', H(m'))$ and Bob's check passes.*

Proof. H is public and keyless; Eve computes $H(m')$ herself. Integrity against an adversary needs a secret: a MAC key or a signing key. □

Password storage

Store H(password), not the password; on login hash the input and compare. A stolen digest is harder to use than plaintext, but *preimage resistance assumes an unknown high-entropy input*: a password from a small dictionary is found by enumeration however good H is. **Dictionary attack**: precompute digests of common passwords; unsalted, the same password gives the same digest for every user, so one table serves all accounts and equal digests reveal shared passwords. **Salt**: a random per-account value stored beside the digest, combined with the password before hashing; it need not be secret, since its job is to make each account's computation *different*: no shared tables, one campaign per user. Salts do not strengthen a weak password; *slow* password hashes (PBKDF2, bcrypt, scrypt, Argon2) raise the per-guess cost; OpenSSL's `-pbkdf2` is such a derivation.

Go deeper: the details an examiner asks about

- Hashing on the client before sending does not remove server-side hashing: the client hash *is* the password and replays.
- "MD5 is broken" matters for signatures (an attacker crafting two documents with one digest) more than for integrity checksums against accidental corruption, where a second preimage is still needed.

Hard questions, answered

Q. Two users share a password. Unsalted vs salted database?
A. Unsalted: identical digests; cracking one cracks both; precomputed tables apply everywhere. Salted: different digests; separate work per account.

Q. Why is $2^{n/2}$ enough for collisions?
A. Among k random digests the number of pairs is $\approx k^2/2$; the expected number of equal pairs reaches 1 when $k^2/2 \approx 2^n$, i.e. $k \approx 2^{n/2}$ (birthday paradox).

5.5 MACs and Digital Signatures

Definition 5.5.1 (MAC; signature)**.** A **message authentication code** uses a shared key: Alice sends $(m, t = \mathrm{MAC}(K, m))$; Bob recomputes and compares; Eve without K cannot forge a valid tag. Constructions: the last CBC block of $E_K(m)$ (sender needed key and message together; expensive), or hash-based: hash plus symmetric encryption of the digest, or hashing message with a secret value; the hash must be cryptographic. MAC gives integrity and shared-key authentication, *no* confidentiality, *no* non-repudiation (both parties hold K; Bob could have made the tag). A **digital signature** uses the sender's private key: $s = \mathrm{Sign}(\mathrm{priv}, H(m))$, verified with the public key; gives origin authentication, integrity and signatory **non-repudiation** (NIST FIPS 186-4); algorithms RSA, DSA, ECDSA; no pre-shared key.

Reading the formula

Why sign the hash: a 15 MB document under 1024-bit RSA would need thousands of slow operations, and signing raw chosen messages exposes the private key to algebraic attacks; the digest is short and fixed-size, so the signature size is independent of the message. The scheme then needs collision resistance: if Eve finds m_1, m_2 with one digest and gets m_1 signed, the signature validates m_2. *The property table*: encryption C only; MAC I + authentication; signature I + authentication + non-repudiation; none gives availability. Confidentiality and authenticity are separate: secrets sent anonymously (C without A), public information from a trusted source (A without C), secure communication (both).

Hard questions, answered

Q. A confidential contract Bob cannot later deny receiving from Alice. Primitives and order?

A. Sign with Alice's private key over $H(m)$ (authenticity, non-repudiation), then encrypt for Bob (hybrid). Sign first so the signature binds the plaintext.

Q. Is MAC $= H(K\|m)$ safe?
A. Not with Merkle–Damgård hashes: length extension lets Eve compute $H(K\|m\|\text{pad}\|m')$ without K. HMAC (or SHA-3) fixes it.

Q. Why can a MAC not be used as evidence in court?
A. The verifier holds the same key as the signer and could have produced the tag; a third party cannot attribute it. Only a signature attributes.

5.6 Certificates and PKI

The recursion: to verify Alice's signature we need her public key; she cannot prove remotely that a key is hers by signing it with that key. Someone else signs a message binding Alice's identity to her key (Bob); Bob's key is vouched for by Clara, Clara's by Don; Don's key "was shipped to us" and is trusted implicitly.

Definition 5.6.1 (Certificate, CA, chain, PKI)**.** A **certificate** contains an identity and a public key and is signed by someone else; each signer is a **certificate authority**; the implicitly trusted one is a **root CA**; the signatures form a **chain of trust** (Alice ← Bob "CA" ← Clara "CA" ← Don "root"). Obtaining one: generate a key pair (private key secret forever); send the public key and identity to the CA as a **certificate signing request** (everything but the signature) with proof of identity; the CA verifies and signs. Format X.509 (RFC 5280). **PKI** (RFC 4949): the hardware, software, people, policies and procedures to create, manage, store, distribute and *revoke* certificates; it includes a **trust store**, the list of root CAs and keys chosen by the OS or application vendor.

Interpretation & discussion

The base case is trust by *distribution*, not cryptography: root keys arrive with the OS. Whoever controls the trust store controls everything above it: organisations add a private root to sign internally; malware that adds a root can sign a certificate for *any* site and mount man-in-the-middle attacks on every HTTPS connection from that machine; some "security" appliances do exactly this to inspect encrypted traffic, which the course calls stupid and dangerous because one compromised CA key breaks confidentiality for everyone behind it. Verification by a browser: the server certificate's signature by the intermediate, the intermediate's by a trusted root, validity dates, name match, revocation status; only then is the server's public key used.

Hard questions, answered

Q. Why can't Alice self-sign to prove her key?
A. Verifying needs her public key, the thing in question; a self-signature only shows the key pair's holder signed. Trust must enter from outside.

Q. CSR vs the CA generating the pair: difference?
A. With a CSR the private key never leaves Alice; the CA attests only the public key. A CA-generated pair means the CA holds the private key, de-

stroying non-repudiation.

Q. A root CA is added to one laptop by malware. What can the attacker do and what remains safe?
A. MITM every HTTPS connection from that laptop (read and alter). Other machines are unaffected; pinned certificates or fingerprint checks may detect it.

5.7 Steganography

Cryptography hides *meaning*; steganography hides *existence*. Simmons (1983) framed it as the *Prisoners' Problem*: Alice and Bob communicate through a warden; the carriers are *stego objects*, the hidden data the *payload*; a *passive* warden only observes (detection problem), an *active* warden alters the objects (robustness problem). Techniques: null ciphers (third letters of each word: "Send Lawyers, Guns, and Money"; first letters of paragraph one and second letters of paragraph two in the WWI telegram: "PERSHING SAILS FROM N.Y. JUNE 1"); text, image, audio, video, network and email steganography; **LSB**: overwrite the least significant bit of each pixel or sample with payload bits, changing each value by at most 1/255 (1 bit per byte of capacity; detectable by LSB statistics; destroyed by recompression). First use: Greece, c. 440 BC. Encrypt the payload first so that detection does not reveal meaning.

Hard questions, answered

Q. Hide 'A' (`01000001`) in pixels `10110010, 10110011, 10110010, 10110011, 10110010, 10110011, 10110010, 10110011`.
A. `10110010, 10110011, 10110010, 10110010, 10110010, 10110010, 10110010, 10110011`.

Q. Which warden defeats LSB trivially and how?
A. Active: re-encode (JPEG, blur, resample); the LSB plane is destroyed, the picture survives.

Summary: Public-key cryptography, integrity, trust

- $N(N-1)/2$ vs $2N$ keys; trapdoor one-way functions; authenticity of public keys becomes the problem.
- RSA: $n = pq$, φ, $ed \equiv 1$; correct by Euler; secure by factoring; hybrid with AES; padding against determinism.
- Hash: preimage / second-preimage / collision ($2^{n/2}$); keyless, so no authenticity; passwords: salt against shared tables, slow hashes against speed.
- MAC (shared key: I + auth) vs signature (private key over $H(m)$: + non-repudiation); encryption gives C only.
- Certificates bind identity to key by a CA's signature; chain to a distributed root; trust store is the root of everything; PKI includes revocation.
- Steganography hides existence; LSB; passive vs active warden.